

ISTQB · CTFL

Certified Tester Foundation Level

v 4.0

PERSONAL STUDY GUIDE

6 Chapters · Glossary · Common Mistakes

Table of Contents

1. Chapter 1 — Fundamentals of Testing

2. Chapter 2 — Testing Throughout the SDLC

3. Chapter 3 — Static Testing

4. Chapter 4 — Test Analysis and Design

5. Chapter 5 — Managing Test Activities

6. Chapter 6 — Test Tools

7. Glossary

8. Common Mistakes

Chapter 1 - Fundamentals of Testing

The big idea: Testing is much more than just clicking around and hoping something breaks. It's a structured, professional discipline that helps build quality into software — not just find bugs at the end.

Learning Objectives for Chapter 1

LO	K	Topic
FL-1.1.1	K1	Identify typical objectives of testing
FL-1.1.2	K2	Differentiate testing from debugging
FL-1.2.1	K2	Exemplify why testing is necessary
FL-1.2.2	K1	Recall the relationship between testing and quality assurance
FL-1.2.3	K2	Distinguish between error, defect, failure, and root cause
FL-1.3.1	K2	Explain the seven testing principles
FL-1.4.1	K2	Summarize the different test activities and tasks
FL-1.4.2	K2	Explain the impact of context on the test process
FL-1.4.3	K2	Differentiate testware that supports test activities
FL-1.4.4	K2	Explain the value of maintaining traceability between the test basis and testware
FL-1.5.1	K2	Explain the role of human psychology in testing
FL-1.5.2	K1	Recall the advantages of the whole team approach
FL-1.5.3	K2	Distinguish the benefits and drawbacks of independence of testing

Exam tip: Chapter 1 is almost entirely K2 — focus on being able to *explain* and *distinguish*, not just name. The only K1 items are FL-1.1.1, FL-1.2.2, and FL-1.5.2. No K3 in this chapter.

What is Testing, really? — Understand (K2)

Testing is a set of activities to **discover defects**, **evaluate quality**, and **provide information** for decisions.

Nine official objectives (K1 — know them all): - Evaluate work products (requirements, designs, code) - Trigger failures and find defects - Ensure required coverage of the test object - Reduce the risk of inadequate software quality - Verify specified requirements have been fulfilled - Verify compliance with contractual, legal, and regulatory requirements - Provide information to stakeholders for informed decisions - Build confidence in the quality of the test object - Validate whether the test object is complete and works as expected

Common misconceptions to avoid: - Testing is **not only** about finding bugs — it also builds confidence the software works - Testing is **not only** about running code — static testing (reviewing docs, designs) counts too - Testing covers both **verification** ("building it right?") and **validation** ("building the right thing?")

Testing vs. Debugging — 💡 Understand (K2)

Activity	Who does it	What it does
Testing	Testers	Finds failures and defects
Debugging	Developers	Finds the cause, fixes it, removes the defect

Debugging process (3 steps): (1) Reproduction of the failure, (2) Diagnosis — finding the root cause, (3) Fixing the defect. After fixing: **confirmation testing** checks the fix resolved the failure; **regression testing** checks nothing else broke. Static testing nuance: static testing directly finds defects without needing to execute code — no failure reproduction step needed.

Why is Testing Necessary? — 💡 Understand (K2)

Testing contributes to project success in three distinct ways: - **Cost-effective defect detection** — removing defects (debugging) is a non-testing activity; testing provides the evidence that triggers it - **Quality evaluation at SDLC phases** — directly evaluates quality at various points and informs release decisions - **Indirect representation of users** — testers ensure user needs are considered throughout the project

The error → defect → failure chain — 💡 Understand (K2)

- **Error:** A human mistake (e.g., developer misunderstands a requirement)
- **Defect:** The flaw introduced by that error into code or document
- **Failure:** Incorrect behavior when that defect is executed
- **Root cause:** The underlying reason the error happened

Not all defects cause failures — some only fail under specific conditions, some may never cause a failure at all.

Testing vs. Quality Assurance (QA) — 🧠 Remember (K1)

- **Testing** is **product-oriented** and **corrective** — finds defects in what was built
- **QA** is **process-oriented** and **preventive** — improves the process to prevent defects

The 7 Testing Principles — 💡 Understand (K2)

#	Principle	What it means in practice
1	Testing shows presence of defects, not their absence	Passing all tests doesn't mean bug-free
2	Exhaustive testing is impossible	Use risk and priorities to focus
3	Early testing saves time and money	The later a defect is found, the more expensive to fix
4	Defects cluster together	A small number of modules contain most defects (Pareto)
5	Tests wear out	Same tests find fewer new defects — vary and update them
6	Testing is context dependent	Banking app testing differs from game testing
7	Absence of defects is a fallacy	Bug-free system that doesn't meet user needs is still a failure

Test Activities & Tasks — 💡 Understand (K2)

Seven activities — often iterative, not strictly sequential:

1. **Test planning** — Define objectives and approach
2. **Test monitoring and control** — Track progress; take corrective actions
3. **Test analysis** — *What* to test? Identify test conditions from the test basis
4. **Test design** — *How* to test? Turn conditions into test cases and define test data
5. **Test implementation** — Prepare test scripts, suites, environment setup
6. **Test execution** — Run tests, log results, report anomalies
7. **Test completion** — Archive testware, write completion report, capture lessons learned

Test Process in Context — 💡 Understand (K2)

Context factors influence strategy, techniques, automation level, coverage, testware detail, and reporting:

Factor	What it includes
Stakeholders	Needs, expectations, requirements, willingness to cooperate
Team members	Skills, knowledge, availability, experience
Business domain	Criticality, risk tolerance, market, legal/regulatory requirements
Technical factors	Technology type, system architecture, tool support
Project constraints	Scope, time, budget, resources
Organizational factors	Organizational structure, existing policies, culture
Software development lifecycle	Sequential, iterative, Agile, DevOps model being used
Tools	Availability, usability, licensing, compliance

Exam tip (K2): An exam question may describe a project scenario and ask how a specific contextual factor should influence the test approach. Know each factor and what it affects.

Testware — what each activity produces — 💡 Understand (K2)

Activity	Output examples
Planning	Test plan, test schedule, entry criteria, exit criteria, risk register
Monitoring & control	Progress reports
Analysis	Test conditions, defect reports (regarding defects in the test basis)
Design	Test cases, coverage items, test environment requirements
Implementation	Test procedures, test scripts/suites, test data, test execution schedule
Execution	Test logs, defect reports
Completion	Test completion report, action items for improvement, change requests, documented lessons learned

Traceability — 💡 Understand (K2)

Good traceability links test cases back to requirements and risks. It helps: - Measure coverage - Analyze the impact of changes - Support audits and governance reporting

Roles in Testing — 💡 Understand (K2)

- **Test management role** — Planning, monitoring, control, and completion; team leadership
- **Testing role** — Analysis, design, implementation, and execution

One person can hold both roles depending on project context.

Essential Skills & Team Approaches — Understand (K2)

Good testers combine: testing knowledge, thoroughness, analytical/critical thinking, creativity, communication skills, and technical knowledge.

Whole Team Approach — everyone is responsible for quality; testers collaborate closely with developers and business reps (from XP).

Exception: safety-critical systems may need higher test independence.

Independence of Testing — independent testers find different defects due to different cognitive biases: 1. Author tests their own work (lowest) 2. Peers from the same team 3. Testers from outside the team (high) 4. Testers from outside the organization (very high)

- **Benefit:** Different perspective catches more defects
- **Drawback:** Risk of isolation, communication problems, developers losing quality responsibility, "us vs. them" mentality

Key Terms Cheat Sheet — Remember (K1)

Term	Definition
Error	A human mistake
Defect	A flaw introduced by an error
Failure	Incorrect behavior caused by executing a defect
Root cause	The fundamental reason a defect exists
Test basis	Everything used to derive test cases (requirements, designs, code, etc.)
Test object	The item being tested
Testware	All artifacts produced by test activities
Verification	Are we building the product right?
Validation	Are we building the right product?

Practice Questions

- What is the difference between error, defect, and failure? Give an example of each.
- List the 7 testing principles and explain any two in your own words.
- What is the difference between testing and QA?
- What is the difference between verification and validation?
- Why is exhaustive testing impossible?
- What are the two principal roles in testing and what does each focus on?
- What are the benefits and drawbacks of test independence?

- What does traceability help with in the test process?

Chapter 2 - Testing Throughout the Software Development Lifecycle

The big idea: Testing doesn't happen at the end. It lives inside every phase of development, adapts to whatever model the team uses, and changes shape depending on the context.

Learning Objectives for Chapter 2

LO	K	Topic
FL-2.1.1	K2	Explain the impact of the chosen SDLC model on testing
FL-2.1.2	K1	Recall good testing practices that apply to any SDLC model
FL-2.1.3	K1	Recall examples of TDD, ATDD, and BDD
FL-2.1.4	K2	Summarize the DevOps benefits for testing
FL-2.1.5	K2	Explain the shift-left approach
FL-2.1.6	K2	Explain how retrospectives can be used for process improvement in testing
FL-2.2.1	K2	Distinguish the different test levels
FL-2.2.2	K2	Distinguish the different test types
FL-2.2.3	K2	Distinguish confirmation testing from regression testing
FL-2.3.1	K2	Summarize the triggers for maintenance testing
FL-2.3.2	K2	Explain the role of impact analysis for maintenance testing

Exam tip: No K3 in this chapter. K1 items are FL-2.1.2 and FL-2.1.3 — just recall, no explanation needed. Everything else requires you to *explain* or *distinguish*.

2.1 Testing in the Context of an SDLC — 💡 Understand (K2)

The chosen SDLC model directly impacts **how** testing is done — not whether it is done.

The three main SDLC model categories — 💡 Understand (K2)

Model	Examples	How testing fits
Sequential	Waterfall, V-model	Testing begins after development; V-model maps each dev stage to a test level
Iterative	Spiral, Prototyping	Testing occurs within each repeated cycle
Incremental	Unified Process	Each small increment is tested before moving to the next

In practice, many real projects combine models — e.g., Agile uses both iterative and incremental approaches. Agile practices like Scrum, Kanban, TDD, BDD, and XP are methods that operate *within* these model types.

The SDLC impacts testing in 5 ways: - Scope and timing of test activities - Level of detail of test documentation - Choice of test techniques and approach - Extent of test automation - Roles and responsibilities of testers

Good testing practices — apply to ANY SDLC model — 🧠 Remember (K1)

- Every development activity has a corresponding test activity
- Different test levels have specific and different test objectives
- Test analysis and design begin during the corresponding development phase
- Testers review work products as soon as drafts are available

Testing as a Driver: TDD, ATDD, and BDD — 🧠 Remember (K1)

Tests are written **before** the code in all three approaches. All three implement the early testing principle, support shift left, and support an iterative development model.

Approach	Who uses it	How it works
TDD (Test-Driven Development)	Developers	Write a failing test → write code to pass it → refactor. TDD was designed to replace extensive upfront software design.
ATDD (Acceptance Test-Driven Development)	Whole team	Tests are derived from acceptance criteria "as part of the system design process" before the feature is built.
BDD (Behavior-Driven Development)	Whole team	Write tests in plain language using Given/When/Then format; test cases "should then be automatically translated into executable tests."

Exam tip (K1): For FL-2.1.3, you only need to recall who uses each approach and its core idea. The "how it works" details in the table are for context — the exam tests recognition, not reproduction of the procedure.

DevOps and Testing — 💡 Understand (K2)

DevOps bridges development, testing, and operations to deliver high-quality software faster.

Benefits for testing: - Fast feedback on code quality via CI/CD pipelines - Automated regression tests run on every commit - Increased visibility on non-functional quality (performance, reliability) - Reduces repetitive manual testing through automation - CI promotes shift left by encouraging developers to submit high-quality code with component tests and static analysis - Automated CI/CD facilitates establishing stable test environments

Even with heavy automation, manual testing — especially from the user's perspective — is still needed.

Shift Left — 💡 Understand (K2)

Shift left means **testing earlier in the SDLC** — not waiting until code is complete.

Good practices to achieve shift left: - Review specifications from a tester's perspective (find ambiguities early) - Write test cases before the code is written - Use CI/CD to get fast feedback on every code commit - Perform static analysis before dynamic testing - Start non-functional testing at the component level, where possible

Shift left may increase early effort and cost, but saves significantly more later.

Important: Stakeholders must be convinced and bought into the concept. Without stakeholder buy-in, the additional upfront effort cannot be justified.

Retrospectives and Process Improvement — 💡 Understand (K2)

Held at end of an iteration, project, or release milestone. Participants include testers, developers, architects, product owners, and business analysts.

Benefits for testing: - Increased test effectiveness and efficiency - Improved quality of testware - Team bonding and learning - Better cooperation between development and testing - Improved quality of the test basis — deficiencies in requirements extent/quality are identified and addressed

2.2 Test Levels and Test Types — 💡 Understand (K2)

Test Levels — the five levels — 💡 Understand (K2)

Level	Focus	Typically performed by
Component testing (unit)	Individual components in isolation	Developers
Component integration testing	Interfaces and interactions between components	Developers / Testers
System testing	End-to-end behavior of the entire system	Independent testers
System integration testing	Interfaces between the system and external systems/services	Testers
Acceptance testing	Validation; demonstrating readiness for deployment	Business / End users

How test levels are distinguished (K2): Test levels differ from each other by: **test object**, **test objectives**, **test basis**, **defects and failures** (types typically found), and **approach and responsibilities**.

Acceptance Testing sub-types: - **UAT:** Real users validate the system meets their needs - **OAT:** Operational readiness — backup, recovery, security - **Contractual/Regulatory:** Verifies compliance with contracts or regulations - **Alpha testing:** Internal users at the developer's site - **Beta testing:** External users in their own environment

Test Types — what aspect is being tested — 💡 Understand (K2)

Type	What it checks
Functional testing	What the system does — functional completeness, correctness, appropriateness
Non-functional testing	How well the system behaves — ISO/IEC 25010 characteristics: Functional suitability, Performance efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability, Safety
Black-box testing	Behavior based on specifications, without knowledge of internal structure
White-box testing	Internal structure — code paths, architecture, data flows

All four test types can be applied at **any** test level.

Note: "Black-box" and "white-box" appear in both Chapter 2 (as test *types*) and Chapter 4 (as test *technique categories*). In Chapter 2 they classify *what the tests are based on* (spec vs. structure); in Chapter 4 they classify *how test cases are derived*. Same terms, related but distinct meaning — don't let it confuse you.

Change-Related Testing — 💡 Understand (K2)

- **Confirmation testing:** Verifies a specific defect has been fixed

- Approach 1: Execute all test cases that previously failed due to the defect
- Approach 2: Add new tests to cover changes made to fix the defect
- When time is short: may be limited to reproducing the failure that triggered the defect report
- **Regression testing:** Ensures changes haven't broken existing functionality

Regression testing grows with every release — it's a strong candidate for automation.

2.3 Maintenance Testing — 💡 Understand (K2)

Triggered when a system already in production needs to change.

Triggers (v4 categories) — 💡 Understand (K2)

Trigger	Examples
Modifications	Planned enhancements, corrective changes (bug fixes), hot fixes
Upgrades or migrations	Moving to a new platform, OS, or environment
Retirement	End of life — data archiving and retrieval testing

Scope depends on: - Degree of risk of the change - Size of the existing system - Size of the change

Impact Analysis — 💡 Understand (K2)

- Determines which parts of the system could be affected by a change
- Defines the scope of regression testing
- Relies on good traceability

Key Terms Cheat Sheet — 🧠 Remember (K1)

Term	Definition
SDLC	Software Development Lifecycle — the process to plan, create, test, and deliver software
Test level	A group of test activities organized together (component, integration, system, acceptance)
Test type	Classification by what is being tested (functional, non-functional, black-box, white-box)
Shift left	Testing earlier in the lifecycle to catch defects sooner
TDD	Tests are written before the code to drive development
BDD	Tests are written in Given/When/Then natural language format
Regression testing	Re-running tests to detect unintended side effects of a change
Confirmation testing	Re-testing after a fix to verify the defect is resolved
Impact analysis	Evaluating which parts of the system are affected by a change

Practice Questions

- What are the three main SDLC model categories? Give one example of each.
- How does the choice of SDLC model impact testing? Name at least three ways.
- What is the difference between TDD, ATDD, and BDD?
- What are the benefits and risks of DevOps for testing?
- What does "shift left" mean and how can you achieve it?
- What is the difference between component testing and component integration testing?
- What is the difference between confirmation testing and regression testing?
- What are the three triggers for maintenance testing in v4?
- Why is regression testing a good candidate for automation?
- What is the purpose of impact analysis in maintenance testing?

Chapter 3 - Static Testing

The big idea: You don't need to run software to find problems in it. Reviewing documents, designs, and code before execution often catches defects earlier, cheaper, and in places dynamic testing simply can't reach.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 3

LO	K	Topic
FL-3.1.1	K1	Recognize types of work products that can be examined by static testing
FL-3.1.2	K2	Explain the value of static testing
FL-3.1.3	K2	Explain the difference between static and dynamic testing
FL-3.2.1	K1	Identify the benefits of early and frequent stakeholder feedback
FL-3.2.2	K2	Summarize the activities of the review process
FL-3.2.3	K1	Recall which responsibilities are assigned to the principal roles when performing reviews
FL-3.2.4	K2	Distinguish between the different review types
FL-3.2.5	K1	Recall the success factors for reviews

Exam tip: No K3 in this chapter. Four K1 items (FL-3.1.1, FL-3.2.1, FL-3.2.3, FL-3.2.5) — pure recall. FL-3.1.3 and FL-3.2.4 require you to *explain differences* and *distinguish* — classic K2.

3.1 Static Testing Basics — 💡 Understand (K2)

- Evaluates a work product **without executing it**
- Done **manually** (reviews) or **with tools** (static analysis — linters, code analyzers in CI)
- Applies to both verification and validation
- Objectives: improve quality, detect defects, assess readability, completeness, correctness, testability, consistency

What Can Be Examined with Static Testing? — 🧠 Remember (K1)

- Almost **any work product** a human can read and understand: requirements, source code, test plans/cases, backlog items, test charters, project docs, contracts, models
- **Static analysis** requires a formal syntax (code, models, structured text)
- **NOT appropriate**: work products humans can't interpret or that can't be tool-analyzed for legal reasons (e.g., 3rd-party executables)

Why Bother? The Value of Static Testing — 💡 Understand (K2)

- Catches defects **early** — before they propagate and become expensive
- Finds things **dynamic testing can't** — unreachable code, design flaws, defects in non-executable docs
- Builds **shared understanding** among the team
- **Cost-effective** — upfront review cost << late-project defect cost

Defects easier to find through static testing — 💡 Understand (K2)

Category	Examples
Requirements defects	Inconsistencies, ambiguities, contradictions, omissions, duplications
Design defects	Inefficient database structures, poor modularization
Coding defects	Undefined values, undeclared variables, unreachable/duplicated code, excessive complexity
Standards deviations	Naming convention violations, coding standard violations
Interface defects	Mismatched number, type, or order of parameters
Security vulnerabilities	Buffer overflows
Test basis gaps	Missing tests for an acceptance criterion

Static vs. Dynamic Testing — 💡 Understand (K2)

Aspect	Static Testing	Dynamic Testing
Needs execution?	No	Yes
How defects are found	Directly	Via failures → root cause analysis
Applicable to	Executable AND non-executable work products	Executable work products only
Rare code paths	Easier to detect	Hard to reach
Quality characteristics	Maintainability, readability	Performance, reliability

3.2 Feedback and Review Process — 💡 Understand (K2)

Why Early and Frequent Stakeholder Feedback Matters — 🧠 Remember (K1)

- Without regular stakeholder involvement, the product drifts from the original vision
- Consequences: costly rework, missed deadlines, blame, project failure
- Frequent feedback: prevents misunderstandings early, keeps the team focused on value, addresses risks proactively

The Review Process — 5 Activities — 🧠 Remember (K1)

Based on **ISO/IEC 20246**:

Step	What happens
1. Planning	Define scope, purpose, exit criteria, effort, and timeframes
2. Review initiation	Ensure all participants have access, understand their role, and are prepared
3. Individual review	Each reviewer independently assesses the work product; logs anomalies, recommendations, and questions
4. Communication and analysis	Anomalies are discussed in a review meeting; decisions made on status, ownership, and follow-up actions
5. Fixing and reporting	The author fixes the defects found; the review leader writes the review report; the work product is accepted once exit criteria are met

Practical note: The size of many work products makes them too large to be covered by a single review. The review process may be invoked multiple times to complete the review for the entire work product.

Roles in a Review — 🧠 Remember (K1)

Role	Responsibility
Manager	Decides what gets reviewed; provides resources (people, time)
Author	Creates and later fixes the work product under review
Moderator (facilitator)	Runs the review meeting; handles mediation, time, and ensures a safe environment
Scribe (recorder)	Records anomalies, decisions, and new findings during the meeting
Reviewer	Performs the actual review; can be a team member, SME, or any stakeholder
Review leader	Owens the review overall; decides who participates, when, and where

Memory tip: Think of a review as a structured meeting with a **boss** (manager), a **creator** (author), a **referee** (moderator), a **note-taker** (scribe), **evaluators** (reviewers), and an **organizer** (review leader).

Review Types — From Least to Most Formal — 💡 Understand (K2)

Type	Formality	Led by	Key characteristics
Informal review	Lowest	Anyone	No defined process; no formal output; just find anomalies
Walkthrough	Low–Medium	Author	Author guides the team through the work product; objectives: evaluating quality and building confidence in the work product, educating reviewers, gaining consensus, generating new ideas, motivating and enabling authors to improve, detecting anomalies; Scribe: Not required (optional — the author may also fill this role); Reviewers might perform an individual review before the walkthrough, but this is not required
Technical review	Medium–High	Moderator	Technically qualified reviewers; objectives: gaining consensus and making decisions on technical problems, detecting anomalies, evaluating quality, building confidence in the work product, generating new ideas, motivating authors to improve
Inspection	Highest	Moderator (NOT the author)	Follows the full defined process; maximum anomaly detection is the main goal; metrics are collected; author cannot be review leader or scribe

Key exam point: In an inspection, the **author cannot act as review leader or scribe** — this is unique to this review type.

Exam tip: The syllabus does NOT mandate a scribe for walkthroughs. Individual review beforehand is also "not required."

Factors affecting required formality: The required level of review formality depends on: the SDLC being followed, the maturity of the development process, the criticality and complexity of the work product, legal or regulatory requirements, and the need for an audit trail.

Success Factors for Reviews — 🧠 Remember (K1)

- Objectives are **clear and measurable** — evaluation of participants is **never** an objective
- The **right review type** is chosen for the work product and context
- Reviews are done on **small chunks** so reviewers stay focused
- **Feedback** is given to stakeholders and authors so they can improve
- Participants have **adequate preparation time**
- **Management supports** the review process
- Reviews are part of the **organization's culture**
- All participants receive **adequate training**
- Meetings are **facilitated effectively**

Key Terms Cheat Sheet — 🧠 Remember (K1)

Term	Definition
Static testing	Evaluating a work product without executing it
Dynamic testing	Testing by executing the software
Static analysis	Tool-assisted evaluation against a formal syntax
Review	Manual evaluation of a work product by one or more people
Inspection	Most formal review type; follows a complete defined process; author cannot be leader or scribe
Technical review	Review led by a moderator with technically qualified reviewers
Walkthrough	Review led by the author
Informal review	Review with no defined process or formal output
Anomaly	Any result that differs from what is expected; identified during a review
Moderator	Ensures the effective running of a review meeting

Practice Questions

- What is static testing and how does it differ from dynamic testing?
- What types of work products can be examined by static testing?
- Name three defect types that are easier to find through static testing.
- What are the five activities in the review process?
- What are the six principal roles in a review? What does each one do?
- What is the most formal review type and what makes it unique?
- What is the main difference between a walkthrough and a technical review?
- Why can't the author be the review leader or scribe in an inspection?
- List four success factors for reviews.

- Why should evaluation of participants never be an objective of a review?

Chapter 4 - Test Analysis and Design

The big idea: How do you decide *what* to test and *how* to test it without just guessing? Test techniques give you a systematic, repeatable way to design test cases that cover what matters — based on specifications, code structure, or your own experience.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 4

LO	K	Topic
FL-4.1.1	K2	Summarize the categories of test techniques and their subcategories
FL-4.2.1	K3	Apply equivalence partitioning to derive test cases for a given scenario
FL-4.2.2	K3	Apply boundary value analysis to derive test cases for a given scenario
FL-4.2.3	K3	Apply decision table testing to derive test cases for a given scenario
FL-4.2.4	K3	Apply state transition testing to derive test cases for a given scenario
FL-4.3.1	K2	Explain statement testing
FL-4.3.2	K2	Explain branch testing
FL-4.3.3	K2	Explain the value of white-box testing
FL-4.4.1	K2	Explain error guessing
FL-4.4.2	K2	Explain exploratory testing
FL-4.4.3	K2	Explain checklist-based testing
FL-4.5.1	K2	Explain the factors that make a good user story (INVEST criteria)
FL-4.5.2	K2	Recall the typical formats for writing acceptance criteria
FL-4.5.3	K3	Use ATDD to derive test cases

Exam tip: The four black-box techniques (FL-4.2.1–4.2.4) are **K3** — the exam will give you a scenario and ask you to produce test cases. Practice deriving them, not just explaining them. White-box techniques (FL-4.3.1–4.3.2) are only K2 — you explain statement and branch coverage, not calculate it.

4.1 The Three Categories of Test Techniques — Understand (K2)

Category	Also known as	Based on	Key characteristic
Black-box	Specification-based	Specified behavior	Tests are independent of implementation — they survive code changes
White-box	Structure-based	Internal structure/code	Tests depend on how the software is built; created after design/implementation
Experience-based	—	Tester's knowledge and skill	Complementary to the other two; finds defects the others miss

Memory tip: Black = you can't see inside the box. White = you can see everything inside. Experience = you've seen similar boxes break before.

4.2 Black-Box Test Techniques — Apply (K3)

4.2.1 Equivalence Partitioning (EP) — Apply (K3)

- Divide input/output data into **partitions** where all values are processed identically
- **One test case per partition** is sufficient
- **Valid partition** = values the system accepts; **Invalid partition** = values the system rejects
- **100% EP coverage** = every partition (valid AND invalid) exercised at least once
- Coverage = partitions exercised ÷ total partitions
- **Multiple inputs:** use **Each Choice coverage** — at least one value from each partition set in at least one test case

Partition types: Partitions may be continuous or discrete, ordered or unordered, finite or infinite. Sources of partitions include not only input parameters but also: configuration items, internal values, time-related values, and interface parameters. Partitions must be non-empty and non-overlapping.

Example: A field accepts ages 18–65. Partitions: under 18 (invalid), 18–65 (valid), over 65 (invalid). You need at least 3 test cases — one per partition.

4.2.2 Boundary Value Analysis (BVA) — Apply (K3)

- Extends EP by targeting the **edges** of ordered partitions — where developers make the most mistakes
- Only applicable to **ordered** partitions (where min/max exist)

Two versions:

Version	Coverage items per boundary	More rigorous?
2-value BVA	Boundary value + its closest neighbor from the adjacent partition	Less
3-value BVA	The boundary value itself, the value immediately below it, and the value immediately above it	More — can catch off-by-one errors 2-value misses

Example: For a range 1–10 using 3-value BVA, test: 0, 1, 2 (lower boundary) and 9, 10, 11 (upper boundary).

100% coverage: Exercise all boundary values (2-value) or all boundary values and their neighbors (3-value).

4.2.3 Decision Table Testing — Apply (K3)

- Maps every **combination of conditions** to expected actions — best for complex business rules
- Each **column** = one decision rule (unique condition combo + resulting actions)
- Notation: **T/F** = condition true/false; **–** = irrelevant; **N/A** = infeasible; **X** = action occurs; **blank** = action does not occur
- **100% coverage** = all feasible columns exercised; Coverage = feasible columns exercised ÷ total feasible columns
- **Strength:** reveals gaps/contradictions in requirements; nothing overlooked
- **Weakness:** rules grow exponentially (2^n) — use a minimized table or risk-based approach

Note: Coverage is measured against the actual columns in the table being used. If the table has been minimized (rules collapsed), coverage is calculated against its actual columns — not against the theoretical 2^n maximum.

Extended-entry decision tables: The examples above show limited-entry tables (conditions are boolean: true/false). Extended-entry tables allow conditions to take multiple values (ranges, partitions, or discrete values) and actions to specify exact output values. They are more compact but harder to read.

4.2.4 State Transition Testing — Apply (K3)

- Tests systems whose behavior depends on **current state**
- **State diagram:** shows all states and valid transitions (**event** [guard condition] / **action**)
- **State table:** rows = states, columns = events; empty cells = invalid transitions
- A test case = a sequence of events producing a sequence of state changes

Three coverage criteria (weakest → strongest):

Coverage	What it exercises	Notes
All states	All states at least once	Weakest — can be achieved without testing all transitions
Valid transitions (0-switch)	All valid transitions — each valid transition exercised at least once, without chaining sequences (0-switch = single steps only)	Most widely used criterion
All transitions	All valid + all invalid transitions	Strongest; required for mission/safety-critical software

Key exam point: Achieving valid transitions coverage guarantees all states coverage. Achieving all transitions coverage guarantees both. Test only **one** invalid transition per test case to avoid defect masking.

4.3 White-Box Test Techniques — 💡 Understand (K2)

4.3.1 Statement Testing — 💡 Understand (K2)

- Coverage items: **executable statements**
- **100% statement coverage** = every executable statement run at least once
- Coverage = statements exercised ÷ total executable statements
- **Limitation:** does NOT guarantee all branches are tested — condition-specific defects may be missed

4.3.2 Branch Testing — 💡 Understand (K2)

- Coverage items: **branches** (transfers of control between nodes in a control flow graph)
- Branches: **unconditional** (straight-line) or **conditional** (if/else, switch, loops)
- **100% branch coverage** = all branches exercised; Coverage = branches exercised ÷ total branches

Critical rule: Branch coverage subsumes statement coverage. If you achieve 100% branch coverage, you automatically achieve 100% statement coverage. The reverse is NOT true.

Limitation: Doesn't detect defects that only appear on a specific path through the code.

4.3.3 The Value of White-Box Testing — 💡 Understand (K2)

- **Strength:** takes the entire implementation into account — finds defects even when the spec is vague or incomplete
- **Weakness:** if a required feature is **missing entirely** (defect of omission), white-box won't detect it
- Usable in both **static testing** (code/pseudocode analysis) and **dynamic testing** (executing code)

Black-box testing alone gives you no measure of actual code coverage. White-box testing provides that objective measurement.

4.4 Experience-Based Test Techniques — 💡 Understand (K2)

4.4.1 Error Guessing — 💡 Understand (K2)

- Uses the tester's knowledge of past failures, typical developer mistakes, and common failure types
 - **Fault attacks:** structured list of likely errors/defects → design tests specifically to expose them
 - Lists come from experience, defect data, or common failure knowledge
-

4.4.2 Exploratory Testing — 💡 Understand (K2)

- Design, execute, and evaluate tests **simultaneously** while learning about the system
 - **Session-based:** time-boxed sessions guided by a **test charter** (contains test objectives); ends with a **debriefing**
 - **Best used when:** specs are few/missing, time pressure is high, or to complement formal techniques
 - More effective with experienced testers who have domain knowledge, curiosity, and analytical thinking
-

4.4.3 Checklist-Based Testing — 💡 Understand (K2)

- Uses a prepared list of test conditions (often phrased as questions) to guide testing
 - Built from experience, user knowledge, and understanding of how software fails
 - Must be **regularly updated** based on defect analysis — checklists can wear out
 - **Exclude:** auto-checkable items, entry/exit criteria items, overly general items
 - **Trade-off:** broad coverage and consistency, but less repeatability than detailed test cases
-

4.5 Collaboration-Based Test Approaches — 💡 Understand (K2)

Focus on **avoiding defects** through shared understanding, not just finding them.

4.5.1 Collaborative User Story Writing — 💡 Understand (K2)

- A **user story** represents a feature valuable to a user or purchaser
- **3 Cs:** Card (the medium), Conversation (how it will be used), Confirmation (acceptance criteria)
- **Standard format:** *"As a [role], I want [goal], so that I can [resulting business value]."*
- **Good user stories follow INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable

Collaboration techniques: User stories are created collaboratively using techniques such as **brainstorming** and **mind mapping**. The collaboration brings together three perspectives: **business** (what value is needed), **development** (what is technically feasible), and **testing** (what could go wrong and what needs to be verified).
If a stakeholder can't imagine how to test a user story, it's a signal the story is unclear or not valuable enough.

4.5.2 Acceptance Criteria — 💡 Understand (K2)

- **Conditions a user story must meet** to be accepted by stakeholders — these are the acceptance test conditions

- Used to: define story scope, reach consensus, describe positive/negative scenarios, enable planning/estimation

Two most common formats:

Format	Example
Scenario-oriented (BDD)	Given [context], When [action], Then [outcome]
Rule-oriented	Bullet point verification list or input-output mapping table

4.5.3 Acceptance Test-Driven Development (ATDD) — ⚙️ Apply (K3)

- Write test cases **before** implementing the user story, using acceptance criteria as the guide
- Steps: (1) **Specification workshop** — team resolves ambiguities; (2) **Test case creation** — based on AC, serve as examples of correct behavior; (3) **Implementation** — developers write code to pass the tests
- Test order: positive (happy path) → negative → non-functional quality characteristics
- Test cases must be in **language understandable by stakeholders**

Test case scoping rule: ATDD test cases must cover all characteristics of the user story and should not go beyond the story. No two test cases should describe the same characteristics.

When captured in an automation framework format, acceptance tests become **executable requirements**.

Term	Definition
Black-box technique	Derives tests from specified behavior; independent of internal structure
White-box technique	Derives tests from internal structure/code
Experience-based technique	Uses tester's knowledge and experience to design tests
Equivalence partitioning	Divides data into partitions processed identically; one test per partition
Boundary value analysis	Tests at the edges of equivalence partitions
Decision table testing	Tests combinations of conditions and their resulting actions
State transition testing	Tests behavior based on system states and transitions between them
Statement coverage	% of executable statements exercised by tests
Branch coverage	% of branches exercised; subsumes statement coverage
Error guessing	Anticipating defects based on experience and failure knowledge
Exploratory testing	Simultaneous design, execution, and evaluation within a time-boxed session
Checklist-based testing	Testing guided by a prepared list of conditions, often as questions
User story	A description of a feature valuable to a user; has Card, Conversation, Confirmation
INVEST	Criteria for a good user story: Independent, Negotiable, Valuable, Estimable, Small, Testable
Acceptance criteria	Conditions a user story must meet to be accepted; test conditions for acceptance tests
ATDD	Test cases written before the user story is implemented, based on acceptance criteria

Practice Questions

- What are the three categories of test techniques and how do they differ?
- In equivalence partitioning, why is one test per partition considered sufficient?
- What is the difference between 2-value BVA and 3-value BVA? When would you use each?
- What does each symbol mean in a decision table (T, F, –, N/A, X)?
- What are the three state transition coverage criteria, from weakest to strongest?
- Why should you test only one invalid transition per test case in state transition testing?
- What does "branch coverage subsumes statement coverage" mean?
- What is the main strength and weakness of white-box testing?
- What is a fault attack in error guessing?
- When is exploratory testing most useful?
- What are the 3 Cs of a user story?

- What does INVEST stand for?
- What are the two most common formats for writing acceptance criteria?
- What happens in the specification workshop in ATDD?
- What is the main difference between defect detection techniques (4.2–4.4) and collaboration-based approaches (4.5)?

Chapter 5 - Managing Test Activities

The big idea: Good testing doesn't just happen — it has to be planned, tracked, and wrapped up properly. This chapter is about the organizational and managerial side of testing: how to define strategy, estimate effort, measure progress, manage risk, and report results.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 5

LO	K	Topic
FL-5.1.1	K2	Exemplify the purpose and content of a test plan
FL-5.1.2	K1	Recognize how a tester adds value to iteration and release planning
FL-5.1.3	K2	Compare entry criteria and exit criteria
FL-5.1.4	K3	Apply estimation techniques to calculate test effort for a defined scenario
FL-5.1.5	K3	Apply test case prioritization for a given scenario
FL-5.1.6	K2	Recall the concepts of the test pyramid
FL-5.1.7	K2	Summarize the testing quadrants and their relationship to test levels and test types
FL-5.2.1	K2	Explain the purpose of test monitoring and control and the difference between them
FL-5.2.2	K2	Summarize the metrics used for testing
FL-5.2.3	K2	Summarize the purpose and content of test reports
FL-5.3.1	K2	Explain how risks influence the scope of testing
FL-5.3.2	K1	Recall risk identification and risk assessment
FL-5.4.1	K3	Apply a defect report for a given scenario
FL-5.4.2	K2	Summarize the content and purpose of a defect report

Exam tip: Three K3 items here — estimation (FL-5.1.4), prioritization (FL-5.1.5), and defect reports (FL-5.4.1). The exam will give you a situation and ask you to produce something: an estimate, a prioritized list, or a defect report. FL-5.1.2 and FL-5.3.2 are K1 — pure recall.

5.1 Test Planning — 💡 Understand (K2)

A **test plan** is a living document describing scope, approach, schedule, and resources for testing.

What a test plan typically includes — 💡 Understand (K2)

Section	Content
Context	Scope, test basis, constraints, test environment
Objectives	What testing aims to achieve
Stakeholders	Roles, responsibilities, communication
Test approach	Strategy, techniques, types of testing
Schedule	Milestones, effort, timelines
Resources	People, tools, environments
Entry and exit criteria	When to start and when to stop testing
Risk and issues	Known risks and mitigation strategies
Metrics	What will be measured to track progress

5.2 Tester's Contribution to Iteration and Release Planning — 🧠 Remember (K1)

- **Release planning:** estimate effort across iterations, analyze risks, define test approach
- **Iteration planning:** analyze user stories, assess testability, identify acceptance criteria, estimate per story, participate in risk identification

5.3 Entry Criteria and Exit Criteria — 💡 Understand (K2)

Entry criteria — when to **START** testing — 💡 Understand (K2)

- Testware is available (test cases, scripts, data)
- Test environment is ready and configured
- The test object has passed a smoke test

Exit criteria — when to **STOP** testing — 💡 Understand (K2)

- Planned tests have been executed
- Coverage level achieved (statements, requirements)
- Defect density is below the defined threshold
- Time or budget has been exhausted (pragmatic exit)

5.4 Estimation Techniques — ⚙️ Apply (K3)

Estimation Techniques — ⚙️ Apply (K3)

Technique	How it works
Estimation based on ratios	Uses metrics from previous similar projects (e.g., ratio of test effort to development effort). Applied to current project data to estimate test effort.
Extrapolation	Uses measurements collected early in the current project to estimate remaining effort. The more data collected, the more accurate the estimate.
Wideband Delphi	Expert-based group technique. Experts independently estimate, share results anonymously, discuss differences, and repeat until consensus is reached. Planning Poker is a variant of Wideband Delphi adapted for Agile.
Three-point estimation	Produces three estimates: most optimistic (a), most likely (m), most pessimistic (b). Final estimate = $(a + 4m + b) / 6$. Reduces bias and accounts for uncertainty.

Exam note: Planning Poker is a **variant of Wideband Delphi**, not a separate technique. Both are named in the syllabus.

5.5 Test Case Prioritization — ⚙️ Apply (K3)

Prioritization determines the order in which tests are run.

Strategies — ⚙️ Apply (K3)

Strategy	Prioritize by
Risk-based	Highest risk areas first — most impact if they fail
Coverage-based	Tests that cover more requirements or code first
Requirements-based	Tests linked to the most critical or important requirements

In practice, risk-based prioritization is the most common and the most effective — it maximizes defect detection in the shortest time when time is limited.

5.6 The Test Pyramid — 💡 Understand (K2)

The test pyramid groups tests by granularity and recommends distribution across levels.

- **Bottom (unit):** many, fast, isolated — cheap to write and maintain

- **Middle (integration)**: tests interactions between components
- **Top (UI/E2E)**: few, slow, brittle — use only where needed

Important: The test pyramid is a model, not a fixed rule. **The number and naming of the layers may differ.**
 The principle: lower layers = more tests, faster, cheaper; higher layers = fewer tests, slower, more expensive.
 The pyramid implies: **automate at the lowest level possible.**

5.7 Testing Quadrants — 💡 Understand (K2)

Two axes: **Business-facing vs. Technology-facing** · **Supporting the team vs. Critiquing the product**

	Supporting the team	Critiquing the product
Technology-facing	Q1: Component tests, component integration tests	Q4: Performance tests, load tests, stress tests, security tests, smoke tests (note: usability tests go in Q3, not Q4)
Business-facing	Q2: Functional tests, story tests, user story tests, prototypes, simulations, API testing	Q3: Exploratory testing, usability testing, user acceptance testing

The quadrants are a **communication tool** — they help teams talk about what kinds of testing are needed and who should do them. They are not prescriptive.

5.8 Risk and Testing — 💡 Understand (K2)

Key concepts — 🧠 Remember (K1)

- **Risk:** potential event/hazard/threat whose occurrence causes an adverse effect. Risk level = likelihood × impact
- **Product risk:** risk that something in the system will fail — drives testing scope
- **Project risk:** risk to the project itself (schedule, budget, team) — managed through project management

Project Risk Categories — 💡 Understand (K2)

- **Organizational:** funding cuts, conflicting priorities, structural changes
- **People:** insufficient skills, staff conflicts, communication issues
- **Technical:** technology failures, tool issues, environment problems
- **Supplier:** third-party delivery failures, supplier going out of business, contract issues

Risk-Based Testing — 💡 Understand (K2)

- **What to test:** high-risk areas get more focus
- **How much:** riskier areas get more test cases
- **When:** high-risk items are tested earlier

- **Two stages:** (1) Risk identification — brainstorm what could go wrong; (2) Risk assessment — evaluate likelihood and impact, then prioritize

5.2.4 Product Risk Control — 💡 Understand (K2)

- **Risk mitigation:** test high-risk areas more thoroughly; use specific techniques; apply multiple independent activities to high-risk items
- **Risk monitoring:** track whether mitigation reduced risk; identify new emerging risks; update the risk register

5.9 Test Monitoring, Test Control, and Test Completion — 💡 Understand (K2)

Test Monitoring — 💡 Understand (K2)

Ongoing activity of checking test progress against the plan.

Common test metrics collected:

Metric	What it tells you
% of test cases executed	How much of the plan is done
% passed / failed / blocked	Quality signal
Defect detection rate	How many defects are being found over time
Defect density by module	Where defects cluster
Requirements / code coverage	How thoroughly the system is exercised
Resource / budget consumption	Is the project on track?

Test Control — 💡 Understand (K2)

Taking corrective actions when monitoring reveals deviations from the plan.

- Reprioritize tests when a critical defect is found
- Adjust schedule or scope if risks materialize
- Change the test approach or add resources

Monitoring without control is just observation. Control is what makes monitoring useful.

Test Progress Reports — 💡 Understand (K2)

- Test period covered; progress against plan; impediments/blockers; planned next activities

Test Completion Reports — 💡 Understand (K2)

- What was tested, how, and with what results; deviations from plan; defect status; testware to archive; lessons learned

5.3.3 Communicating the Status of Testing — 💡 Understand (K2)

Option	When to use
Verbal communication	Informal, immediate updates; useful for daily collaboration
Dashboards	Real-time visual summaries of test progress, coverage, and defect status; suitable for distributed teams and executives
Electronic communication	Emails or automated messaging for specific events (e.g., test run completed, critical defect found)
Test progress reports	Formal periodic reports during test execution; tracks progress against plan
Test completion reports	Produced at major milestones (end of a test level, end of iteration, release); includes summary of what was tested, results, remaining risks

Exam tip (K2): A question may describe a stakeholder's information need and ask which communication channel is most appropriate. Match the formality and frequency of the channel to the stakeholder's needs.

5.4 Configuration Management — 💡 Understand (K2)

CM identifies, controls, and tracks work products and versions throughout the project lifecycle.

- **Configuration item:** any work product under CM (test cases, scripts, data, environment specs, plans)
- **Baseline:** formally approved version of an item — a known, stable state; changes require formal change control
- **CM and testing:** ensures testers use correct versions; enables reproducibility; supports impact analysis

A good defect report includes —  Apply (K3)

Field	Purpose
Unique ID	Tracking
Summary	Short description of the failure
Date and author	Who found it and when
Environment	OS, browser, version, configuration
Steps to reproduce	Exact sequence to reproduce the failure
Expected result	What should have happened
Actual result	What actually happened
Severity	Impact on the system (Critical, Major, Minor, Trivial)
Priority	How urgently it needs to be fixed
Status	Current state in the lifecycle
Attachments	Logs, screenshots, videos
Context of the defect	Test phase, environment, test technique in use when the defect was found
References	Links to the test case that found the defect, related requirements, or related defects

Severity vs. Priority: A typo in a login button label is low severity but possibly high priority (it's the first thing every user sees). A crash in a rarely-used feature might be high severity but low priority.

Term	Definition
Test plan	Document describing the scope, approach, schedule, and resources for testing
Entry criteria	Conditions that must be met before a test activity begins
Exit criteria	Conditions that determine when a test activity is complete
Risk	A potential event, hazard, threat, or situation whose occurrence causes an adverse effect
Product risk	Risk that a system component fails to meet expected behavior
Project risk	Risk to the project schedule, budget, or resources
Risk-based testing	Using risk as the guide for prioritizing what, how much, and when to test
Test pyramid	Model recommending more unit tests than integration or UI tests; number and naming of layers may vary
Testing quadrants	Framework categorizing tests by audience (business vs. technology) and purpose (guiding vs. critiquing)
Test monitoring	Checking test progress against the plan
Test control	Taking corrective action when deviations are found
Defect report	A document capturing a failure and all information needed to reproduce, track, and resolve it
Severity	The impact of a defect on the system
Priority	The urgency with which a defect should be fixed
Configuration management	Tracking versions of testware to ensure consistency and reproducibility
Wideband Delphi	Expert-based estimation technique using anonymous rounds of estimation and discussion until consensus
Three-point estimation	Estimation using optimistic, most likely, and pessimistic values: $(a + 4m + b) / 6$
Risk mitigation	Actions taken to reduce the likelihood or impact of identified risks
Risk monitoring	Tracking risks throughout the project and updating the risk register accordingly

Practice Questions

- What is the purpose of a test plan and what does it typically include?
- What is the difference between entry criteria and exit criteria? Give one example of each.
- What are the four estimation techniques named in CTFL v4.0.1? How does three-point estimation work?
- What is the relationship between Planning Poker and Wideband Delphi?
- What is risk-based test prioritization and why is it the most common strategy?

- What is the test pyramid and what does it recommend about automation? Why can the number of layers vary?
- What are the four testing quadrants and what axes define them?
- Which quadrant does usability testing belong to, and why does it matter?
- What is the difference between product risk and project risk?
- What are the four categories of project risk named in the syllabus?
- What are the two stages of risk-based testing (identification and assessment)?
- What are the two activities of product risk control?
- What is the difference between test monitoring and test control?
- What should a test completion report include?
- What are the five communication options for reporting test status? When would you use a dashboard vs. a verbal update?
- What is the difference between severity and priority in defect management?
- Why does configuration management matter for testing? What is a baseline?
- What are the two fields added to defect reports beyond the basic list (context and references)?
- How do entry/exit criteria relate to Definition of Ready and Definition of Done in Agile?
- What metrics would you collect to monitor test progress?

Chapter 6 - Test Tools

The big idea: The right tool makes testing faster, more consistent, and more thorough — but tools are not a silver bullet. Knowing what categories of tools exist, what they actually do, and how to choose them wisely is what this chapter is about.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 6

LO	K	Topic
FL-6.1.1	K2	Explain how different types of test tools support testing
FL-6.2.1	K1	Recall the benefits and risks of test automation

Exam tip: Chapter 6 is the shortest chapter and has **no K3**. The highest level is K2 (FL-6.1.1). FL-6.2.1 is K1 — recall, not explanation. Don't over-invest time here.

6.1 Tool Support for Testing — 💡 Understand (K2)

Tools are categorized by the **activity they support** — many modern tools cover multiple categories.

Key exam point (FL-6.1.1): You need to be able to match a tool category to the test activity it supports. The question format is typically: "which type of tool would support activity X?"

Management and Reporting Tools — 💡 Understand (K2)

Tool type	What it supports
Test management tools	Planning, monitoring, and controlling test activities; managing test cases, test suites, and execution schedules
Defect tracking tools	Logging, tracking, and managing defects through their lifecycle
Configuration management tools	Tracking versions of the software under test and testware; ensuring reproducibility
Requirements management tools	Traceability between requirements and test cases

Static Testing Tools — 💡 Understand (K2)

Tool type	What it supports
Static analysis tools	Analyzing source code without executing it — detecting coding standard violations, security vulnerabilities, unreachable code, undefined variables
Spelling and grammar checkers	Reviewing written work products (requirements, test cases) for language quality

Test Design and Implementation Tools — 💡 Understand (K2)

Tool type	What it supports
Test data generation tools	Creating input data for tests automatically — useful for boundary values, large datasets, and masked production data
Test case design tools	Generating test cases from models or specifications (used in model-based testing)
Model-based testing tools	Automatically deriving test cases from a behavioral model of the system

Test Execution and Coverage Tools — 💡 Understand (K2)

Tool type	What it supports
Test execution tools (test automation frameworks)	Running test scripts automatically; comparing actual vs. expected results; logging results
Coverage measurement tools	Measuring how much of the code or requirements has been exercised by tests (e.g., statement coverage, branch coverage)

- **Data-driven testing:** test script is separated from test data (spreadsheet, database) — same script, many data sets
- **Keyword-driven testing:** test logic expressed as readable keywords (e.g., `Login`, `ClickButton`) — less technical to maintain

Non-Functional Testing Tools — 💡 Understand (K2)

Tool type	What it supports
Performance testing tools	Simulating load, measuring response times, identifying bottlenecks (load testing, stress testing, endurance testing)
Security testing tools	Scanning for vulnerabilities, testing authentication and authorization, penetration testing support
Accessibility testing tools	Verifying that the system can be used by people with disabilities

DevOps and Collaboration Tools — 💡 Understand (K2)

Tool type	What it supports
CI/CD pipeline tools	Automatically triggering test execution on every code commit; integrating testing into the build process
Version control / SCM tools	Managing source code and testware versions; enabling branching and parallel development
Collaboration tools	Communication, shared test documentation, remote team coordination, and shared decision-making

Scalability and Deployment Standardization Tools — 💡 Understand (K2)

Tool type	What it supports
Virtual machines and containerization tools (e.g., Docker, Kubernetes)	Supporting scalability and consistent test environment provisioning across teams and pipelines

General-Purpose Tools — 💡 Understand (K2)

Tool type	What it supports
General-purpose tools (e.g., spreadsheets)	Any other tool that assists in testing — spreadsheets are a valid, if basic, test management tool for small projects

6.2 Benefits and Risks of Test Automation — 🧠 Remember (K1)

Automation is most valuable for **repetitive, stable, high-volume testing** — not for everything.

Key principle: Simply acquiring a tool does not guarantee success. Each new tool requires effort to achieve real and lasting benefits — including tool introduction, ongoing maintenance, and team training.

Benefits of Test Automation — 🧠 Remember (K1)

Benefit	Why it matters
Reduces repetitive manual work	Frees testers to focus on exploratory and higher-value testing
Prevents simple human errors	A script executes exactly the same way every time
Objective assessment	Coverage metrics and results are measured consistently
Easy access to test status info	Results dashboards and logs are generated automatically
Reduced test execution time	Tests can run in parallel, overnight, or on every commit
More time for testers to design new, deeper, and more effective tests	Automation handles repetitive regression work, freeing testers to invest in better test design

Risks of Test Automation — 🧠 Remember (K1)

Risk	What can go wrong
Unrealistic expectations	Tools don't find bugs automatically — they only check what they are told to check
Underestimated effort	Automating tests takes significant time to set up, script, and maintain
Using the wrong tool	Not every tool fits every technology stack or test type
Over-reliance on automation	Automated tests can't replace human judgment, especially for UX and exploratory testing
Maintenance burden	When the system changes, test scripts break and must be updated
Vendor dependency	The tool vendor may go out of business, retire the tool, sell it to a different vendor, or provide poor support
Open-source abandonment	An open-source tool may be abandoned, with no further updates or security patches
Platform incompatibility	The automation tool may not be compatible with the development platform, framework, or technology under test
Regulatory non-compliance	The chosen tool may not comply with regulatory requirements or safety standards applicable to the project

The key insight: Automation is a multiplier — it multiplies the efficiency of good testing, but it also multiplies the problems of bad testing. Bad tests automated at scale are worse than no automation.

Term	Definition
Test automation	Using tools to execute tests and compare actual results to expected results automatically
Test automation framework	A set of rules, libraries, and tools that provide the structure for automated test scripts
Data-driven testing	A test automation approach where test data is stored externally and fed into scripts; same script runs with different data sets
Keyword-driven testing	A test automation approach where test actions are expressed as readable keywords, separating test logic from implementation
Static analysis tool	A tool that analyzes source code without executing it, detecting defects and standard violations
Test management tool	A tool that supports planning, monitoring, controlling, and reporting on test activities
Performance testing tool	A tool that simulates load and measures system response times and behavior under stress
Model-based testing	An approach where test cases are automatically generated from a behavioral model of the system
CI/CD	Continuous Integration / Continuous Delivery — a DevOps practice where code changes trigger automated builds and tests
Coverage measurement tool	A tool that measures how much of the code or requirements is exercised by a test suite

Practice Questions

- What are the main categories of test tools? Give one example of what each supports.
- What is the difference between data-driven testing and keyword-driven testing?
- Name three benefits and three risks of test automation.
- Why does test automation require ongoing maintenance effort?
- What factors should be considered when selecting a test tool?
- What is a proof of concept in the context of tool selection, and why is it recommended?
- Why is a passing automated test suite not a guarantee of software quality?
- What is the difference between a static analysis tool and a test execution tool?
- How do CI/CD tools connect to the concept of shift left from Chapter 2?
- Which type of tool would you use to measure branch coverage after running your test suite?

ISTQB CTFL — Glossary

All key terms from Chapters 1–6, sorted alphabetically. Each entry shows the chapter(s) where it is tested and the cognitive level.

How to use: Read the term, try to recall the definition, then check it. Focus extra time on K2 terms (you need to *explain* them, not just recall) and K3 terms (you need to *apply* them).

A

Acceptance criteria Ch4 💡 K2 The conditions a user story must meet to be accepted by stakeholders. They define the scope of the story, guide acceptance testing, and are typically written in scenario-oriented (Given/When/Then) or rule-oriented format.

Acceptance testing Ch2 💡 K2 The test level focused on validation — confirming the system is ready for deployment. Performed by business representatives or end users. Includes UAT, OAT, contractual/regulatory testing, alpha, and beta testing.

Alpha testing Ch2 💡 K2 A type of acceptance testing performed by internal users at the developer's site, before release.

Anomaly Ch3 🧠 K1 Any result or condition that deviates from expectations, discovered during a review. May or may not be a defect — needs to be investigated.

ATDD — Acceptance Test-Driven Development Ch2 Ch4 ⚙️ K3 An approach where test cases are derived from acceptance criteria as part of the system design process, before the user story is implemented. Steps: specification workshop → test case creation → implementation. Test cases must cover all characteristics of the user story and must not go beyond it. Closely related to BDD.

B

BDD — Behavior-Driven Development Ch2 🧠 K1 A test-first approach where tests are written in natural language using the Given/When/Then format, making them understandable to the whole team including non-technical stakeholders.

Beta testing Ch2 💡 K2 A type of acceptance testing performed by external users in their own environment, after release.

Black-box technique Ch4 💡 K2 A test technique that derives test cases from the specified behavior of the software — without knowledge of its internal structure. Tests survive code changes because they are specification-based. See also: *Equivalence Partitioning*, *Boundary Value Analysis*, *Decision Table Testing*, *State Transition Testing*.

Black-box testing Ch2 💡 K2 A test type where tests are based on the specification or specified behavior of the system, without knowledge of internal structure. Can be applied at any test level.

Boundary Value Analysis (BVA)   A black-box technique that extends equivalence partitioning by specifically targeting the boundaries of ordered partitions. Two versions: 2-value BVA (boundary + nearest neighbor) and 3-value BVA (boundary + value immediately below + value immediately above). Exploits the fact that developers make the most mistakes at the edges of ranges.

Branch   A transfer of control between nodes in a control flow graph. Branches can be unconditional or conditional (e.g., from an if/else or switch statement).

Branch coverage   The percentage of branches exercised by a test suite. 100% branch coverage guarantees 100% statement coverage (subsumption), but not vice versa. Stronger than statement coverage.

Branch testing   A white-box technique that designs test cases to exercise all branches in the code. Corresponds to branch coverage.

C

Checklist-based testing   An experience-based technique where tests are guided by a prepared list of conditions, often phrased as questions. Checklists should be regularly updated based on defect analysis to prevent them from wearing out.

Component integration testing   A test level focused on testing the interfaces and interactions between integrated components. Usually performed by developers or testers after component testing.

Component testing   The test level focused on individual components tested in isolation. Also called unit testing. Typically performed by developers.

Confirmation testing   Testing performed after a defect has been fixed, to verify that the specific defect no longer causes a failure. Also called re-testing.

Configuration baseline   A formally approved version of a configuration item, from which changes are controlled. Represents a known, stable state of a work product. Any change to a baseline must go through a formal change control process.

Configuration item   Any work product placed under configuration management — e.g., test plans, test cases, test scripts, test data, or test environment specifications.

Configuration management (CM)   The discipline of identifying, controlling, and tracking work products and their versions throughout the project lifecycle. Ensures testers are always working with the correct version of the test basis and testware. Enables reproducibility of test results and supports impact analysis. Key concepts: configuration item, baseline, change control.

CI/CD — Continuous Integration / Continuous Delivery   A DevOps practice where every code change automatically triggers a build and a test run. Integrates testing directly into the development pipeline, enabling fast feedback and supporting shift left. CI/CD tools connect developer commits to test results without manual intervention.

Coverage measurement tool   A tool that measures how much of the code or requirements has been exercised by a test suite. Produces objective metrics such as statement coverage or branch coverage percentages.

Coverage criterion   A rule or target used to determine whether a test suite is sufficient. Examples: all partitions exercised (EP), all branches exercised (branch coverage), all valid transitions exercised (state transition).

D

Data-driven testing Ch6  K1 A test automation approach where the test script is separated from the test data. Data is stored externally (e.g., a spreadsheet or database) and fed into the script at runtime, allowing the same script to run with many different data sets.

Decision table testing Ch4  K3 A black-box technique used to test combinations of conditions and their resulting actions. Each column in the table represents one rule (unique combination of conditions). Best suited for testing business logic with complex conditional behavior.

Defect Ch1  K1 A flaw or imperfection in a work product (code, document, design) introduced by a human error. When executed, a defect may cause a failure — or may not, depending on the conditions.

Defect management Ch5  K3 The process of tracking defects from discovery to closure. Involves logging, classifying, prioritizing, fixing, retesting, and closing defects. The exam tests the content of a defect report (FL-5.4.1/5.4.2).

Defect report Ch5  K3 A document that records a defect, containing all information needed to reproduce, track, and resolve it. Key fields: unique ID, summary, date/author, environment, steps to reproduce, expected result, actual result, severity, priority, status, attachments.

Definition of Done (DoD) Ch5  K2 Agile equivalent of exit criteria — the conditions that must be met for a user story or iteration to be considered complete.

Definition of Ready (DoR) Ch5  K2 Agile equivalent of entry criteria — the conditions that must be met before work on a user story or iteration can begin.

Dynamic testing Ch1 Ch3  K2 Testing performed by executing the software. Defects are revealed through failures — the system must run. Contrast with static testing.

E

Entry criteria Ch5  K2 Conditions that must be satisfied before a test activity begins. Prevents wasted effort on testing when the system is not ready. In Agile: Definition of Ready (DoR).

Equivalence partitioning (EP) Ch4  K3 A black-box technique that divides input (or output) data into partitions where all values are expected to be treated identically by the system. One test case per partition is sufficient. Coverage = partitions exercised ÷ total partitions. Partitions may be continuous or discrete, ordered or unordered, finite or infinite. Sources include not only input parameters but also configuration items, internal values, time-related values, and interface parameters. Partitions must be non-empty and non-overlapping.

Estimation based on ratios Ch5  K2 A test effort estimation technique that uses metrics from previous similar projects — e.g., the ratio of test effort to development effort — and applies those ratios to current project data.

Extended-entry decision table Ch4  K2 A decision table variant where conditions can take multiple values (ranges, partitions, or discrete values) rather than just true/false. Actions may also specify exact output values. More compact than limited-entry tables but harder to read. Contrast with: *limited-entry decision table*.

Extrapolation Ch5  K2 A test effort estimation technique that uses measurements collected early in the current project to estimate the remaining effort. The more data points collected, the more accurate the estimate becomes.

Error Ch1  K1 A human mistake — for example, a developer misunderstanding a requirement. An error introduces a defect into a work product.

Error guessing Ch4  K2 An experience-based technique that uses the tester's knowledge of how software typically fails to anticipate and target likely defects. Can be structured as a *fault attack* using a list of likely errors.

Exit criteria Ch5  K2 Conditions that determine when a test activity is sufficiently complete. May include coverage targets, defect density thresholds, or time/budget exhaustion. In Agile: Definition of Done (DoD).

Experience-based technique Ch4  K2 A category of test techniques based on the tester's knowledge, intuition, and experience. Complements black-box and white-box techniques. Includes error guessing, exploratory testing, and checklist-based testing.

Exploratory testing Ch4  K2 An experience-based technique where the tester simultaneously designs, executes, and evaluates tests while learning about the system. Uses a *test charter* to guide each time-boxed session. Most effective when specs are incomplete or time is limited.

F

Failure Ch1  K1 The visible, observable incorrect behavior of a system caused by the execution of a defect. Not all defects cause failures — some require specific conditions to trigger.

Fault attack Ch4  K2

A structured form of error guessing: the tester creates a list of likely errors and defects, then designs tests specifically to expose them.

Functional testing Ch2  K2 A test type that evaluates *what* the system does — its functional completeness, correctness, and appropriateness. Can be applied at any test level.

I

Impact analysis Ch2  K2

The process of evaluating which parts of a system are affected by a proposed change. Used before maintenance testing to define the scope of regression testing. Relies on good traceability.

Informal review Ch3  K1 The least formal review type. No defined process, no required output. Goal is simply to find anomalies through a casual inspection.

Inspection Ch3  K1 The most formal review type. Follows a complete defined process based on ISO/IEC 20246. Maximum anomaly detection is the primary goal; metrics are collected. The author **cannot** act as review leader or scribe — unique to this review type.

INVEST Ch4  K2 An acronym defining the characteristics of a good user story: **I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, **T**estable.

K

Keyword-driven testing Ch6  K1 A test automation approach where test actions are expressed as readable keywords (e.g., `Login`, `ClickButton`, `VerifyText`). The keywords are implemented separately, allowing non-programmers to write and maintain tests without scripting knowledge.

M

Maintenance testing Ch2  K2 Testing performed on a system already in production when it needs to change. Triggered by modifications (enhancements, bug fixes, hot fixes), upgrades/migrations (new platform, OS, environment), or retirement (end-of-life data archiving).

Model-based testing Ch6  K1 A test approach where test cases are automatically derived from a behavioral model of the system. The model (e.g., a state machine or decision table) serves as the test basis; a tool generates test cases from it. Reduces manual test design effort for complex systems.

Moderator Ch3  K1 The role responsible for running a review meeting effectively. Handles facilitation, mediation, and time management, and ensures a safe environment for feedback. Required in technical reviews and inspections; optional in walkthroughs.

N

Non-functional testing Ch2  K2 A test type that evaluates *how well* the system performs — covering performance, security, usability, reliability, maintainability, portability, and safety. Can be applied at any test level.

O

OAT — Operational Acceptance Testing Ch2  K2 A type of acceptance testing focused on operational readiness: backup and recovery, security, performance under load, and other operational characteristics.

P

Planning Poker Ch5  K1 An Agile estimation technique where team members independently select a card (representing their estimate) and reveal all cards simultaneously. Differences are discussed until consensus is reached. Planning Poker is a **variant of Wideband Delphi** adapted for Agile teams — it is not a separate estimation technique.

Priority Ch5  K2 How urgently a defect needs to be fixed, from a business perspective. High priority means it must be fixed soon — regardless of severity. A cosmetic bug on the login screen may be low severity but high priority.

Product risk Ch5  K2 The risk that a system component or feature will fail to meet expected behavior or quality. Product risks drive the scope and depth of testing.

Performance testing tool Ch6  K1 A tool that simulates concurrent user load on a system and measures response times, throughput, and resource consumption under stress. Used for load testing, stress testing, and endurance testing.

Project risk Ch5  K2 The risk to the project itself — schedule, budget, resources, or team. Managed through project management, not testing.

Q

Quality assurance (QA)  Ch1  K1 A process-oriented, preventive activity that improves the development process to prevent defects from occurring. Distinct from testing, which is product-oriented and corrective. QA sets up the good process; testing checks the result.

R

Regression testing  Ch2  K2 Re-running tests after a change to detect unintended side effects — ensuring that existing functionality has not been broken. Grows with every release and is a strong candidate for automation.

Review  Ch3  K1 A form of static testing where one or more people manually evaluate a work product. Types range from informal to formal (inspection). Goal: find anomalies, improve quality, build shared understanding.

Review leader  Ch3  K1 The role responsible for organizing the overall review: deciding who participates, when, and where. Cannot be the same person as the scribe or author in an inspection.

Risk  Ch5  K1 A potential event, hazard, threat, or situation whose occurrence causes an adverse effect. Characterized by its likelihood and impact. Risk level = likelihood × impact.

Risk assessment  Ch5  K1 The second stage of risk-based testing: evaluating identified risks by their likelihood and impact to determine which areas need the most testing attention.

Risk identification  Ch5  K1 The first stage of risk-based testing: brainstorming with stakeholders to identify what could go wrong in the system.

Risk-based testing  Ch5  K2 A testing approach that uses risk as the primary guide for deciding what to test, how much to test, and when to test. Ensures the highest-risk areas receive the most testing attention — especially important when time is limited.

Root cause  Ch1  K1 The fundamental underlying reason why an error was made — e.g., a misunderstood requirement, lack of training, or unclear documentation. Fixing root causes prevents future defects.

S

Scribe  Ch3  K1 The role responsible for recording anomalies, decisions, and new findings during a review meeting. Mandatory in inspections. Optional in walkthroughs — the author may fill this role.

Severity  Ch5  K2 The impact of a defect on the system — how badly it affects functionality, data, or operations. Classified as Critical, Major, Minor, or Trivial. Not the same as priority.

Shift left  Ch2  K2 The practice of moving testing activities earlier in the SDLC — performing test analysis, design, and even execution as early as possible to find defects when they are cheapest to fix.

SDLC — Software Development Lifecycle  Ch2  K1 The process used to plan, create, test, and deliver software. The choice of SDLC model (sequential, iterative, incremental) directly impacts how testing is structured and when it occurs.

State transition testing  Ch4  K3 A black-box technique that tests a system's behavior based on its states and the transitions between them. Coverage criteria: all states (weakest), all valid transitions / 0-switch

(most common), all transitions including invalid (strongest).

Statement coverage Ch4 💡 K2 The percentage of executable statements exercised by a test suite. 100% statement coverage does not guarantee all branches are tested. Weaker than branch coverage.

Statement testing Ch4 💡 K2 A white-box technique that designs test cases to exercise executable statements. Corresponds to statement coverage.

Static analysis Ch3 Ch6 💡 K2 Tool-assisted evaluation of a work product against a formal syntax — without executing it. Examples: linters, code analyzers. Finds issues like undefined variables, unreachable code, and coding standard violations. In Ch6, static analysis tools are a named tool category that supports static testing activities.

Static testing Ch1 Ch3 💡 K2 Testing performed without executing the software. Includes both manual reviews and automated static analysis. Can find defects in non-executable work products (requirements, designs, test plans) that dynamic testing cannot reach.

System integration testing Ch2 💡 K2 A test level focused on the interfaces between the system under test and external systems or services. Tests interactions at the system boundary.

System testing Ch2 💡 K2 A test level that tests the end-to-end behavior of the entire system against its specified requirements. Typically performed by independent testers.

T

Test automation Ch6 🧠 K1 The use of tools to execute tests and compare actual results to expected results automatically, without manual intervention. Most effective for repetitive, stable, high-volume testing. Not a replacement for human judgment and exploratory testing.

Test automation framework Ch6 🧠 K1 A set of rules, libraries, and tools that provide the structure and conventions for writing, organizing, and running automated test scripts. Examples include data-driven and keyword-driven frameworks.

Test management tool Ch6 🧠 K1 A tool that supports the planning, monitoring, controlling, and reporting of test activities. Typically manages test cases, test suites, test execution schedules, and defect tracking in one place.

TDD — Test-Driven Development Ch2 🧠 K1 A development practice where developers write a failing unit test first, then write code to make it pass, then refactor. Implements the early testing principle.

Technical review Ch3 💡 K2 A semi-formal review type led by a trained moderator with technically qualified reviewers. Focus is on achieving consensus on technical issues and detecting anomalies. Objectives include gaining consensus on technical problems, generating new ideas, evaluating quality, and building confidence in the work product.

Test basis Ch1 🧠 K1 All the information used to derive test cases — requirements, user stories, designs, code, risk assessments, etc. Good traceability links test cases back to the test basis.

Test charter Ch4 💡 K2 A document used in exploratory testing to define the objectives of a time-boxed test session. Guides exploration without over-constraining it.

Test completion report Ch5 💡 K2 A report produced at the end of a test phase or project. Summarizes what was tested, results, deviations from the plan, defect status, testware to archive, and lessons learned.

Test control Ch5 💡 K2 The activity of taking corrective actions when test monitoring reveals deviations from the test plan — such as reprioritizing tests, adjusting scope, or adding resources.

Test level Ch2 💡 K2 A group of test activities organized and managed together, with a specific focus and test basis. The five levels: component, component integration, system, system integration, and acceptance testing.

Test monitoring Ch5 💡 K2 The ongoing activity of checking actual test progress against the plan, using metrics. Enables test control when deviations are found.

Test object Ch1 🧠 K1 The specific work product being tested — e.g., a component, a system, a document, or a configuration.

Test plan Ch5 💡 K2 A document that describes the objectives, scope, approach, schedule, and resources for a test effort. A living document updated as the project evolves.

Test progress report Ch5 💡 K2 A report that communicates the current testing status to stakeholders. Covers the period, progress vs. plan, impediments, and planned next steps.

Test pyramid Ch5 💡 K2 A model recommending a distribution of tests across levels: many fast, cheap tests at the bottom (closer to code), fewer at mid-levels, and very few slow/brittle tests at the top (closer to UI). Encourages automating at the lowest effective level. **The number and naming of layers may differ** — the principle (more tests at lower levels) matters more than any specific layer structure.

Three-point estimation Ch5 💡 K2 A test effort estimation technique that produces three estimates: most optimistic (a), most likely (m), most pessimistic (b). Final estimate = $(a + 4m + b) / 6$. Reduces bias and accounts for uncertainty by weighting the most likely value four times.

Test type Ch2 💡 K2 A classification of testing by what aspect of the system is being tested: functional, non-functional, black-box, or white-box. All four types can be applied at any test level.

Testware Ch1 💡 K2 All artifacts produced by test activities — including test plans, test cases, test scripts, test data, test logs, defect reports, and completion reports.

Testing quadrants Ch5 💡 K2 A framework (Brian Marick) for categorizing test types by two axes: technology-facing vs. business-facing, and supporting the team vs. critiquing the product. Q1 (tech, support): component/component integration tests. Q2 (business, support): functional tests, story tests, API testing. Q3 (business, critique): exploratory testing, usability testing, UAT. Q4 (tech, critique): performance, load, stress, security, smoke tests — **except usability tests**, which belong in Q3.

U

UAT — User Acceptance Testing Ch2 💡 K2 A type of acceptance testing where real users validate that the system meets their needs and is ready for use in production.

User story Ch4 💡 K2 A description of a feature that delivers value to a user or purchaser. Written as: "*As a [role], I want [goal], so that [business value].*" Has three components: Card, Conversation, Confirmation (the 3 Cs). Should follow the INVEST criteria.

V

Validation Ch1 🧠 K1 Confirming that a product meets the needs of the user and other stakeholders. Asking: "*Are we building the right product?*" Focuses on fitness for purpose.

Verification Ch1 🧠 K1 Confirming that a work product meets its specified requirements. Asking: "*Are we building the product right?*" Focuses on conformance to specifications.

W

Walkthrough Ch3 🧠 K1 A review type led by the author, who guides participants through the work product. Aims to educate, gain consensus, and detect anomalies. Individual review beforehand is **not required**. A scribe is **optional** (the author may fill this role). Objectives include: evaluating quality, building confidence, educating reviewers, gaining consensus, generating new ideas, motivating authors.

Wideband Delphi Ch5 💡 K2 A named CTFL v4.0.1 estimation technique based on expert consensus. Experts independently estimate effort, results are shared anonymously, differences are discussed, and the process repeats until consensus is reached. Planning Poker is a variant of Wideband Delphi adapted for Agile teams.

White-box technique Ch4 💡 K2 A test technique that derives test cases from the internal structure of the software — code paths, branches, data flows. Tests depend on the implementation. Includes statement testing and branch testing.

White-box testing Ch2 💡 K2 A test type where tests are based on the internal structure or implementation of the system. Can be applied at any test level.

ISTQB CTFL — Common Mistakes

Typical exam traps, easily confused concepts, and tips to avoid falling for them.

Easily Confused Concepts

Error vs. Defect vs. Failure vs. Root Cause Ch1

Term	What it is	Who creates it
Error	A human mistake	Developer, analyst, tester
Defect	The flaw the error introduced into the work product	Caused by the error
Failure	The visible incorrect behavior when the defect is executed	Caused by the defect
Root cause	The underlying reason the error was made	Process or environment issue

Trap: "A defect always causes a failure." — FALSE. A defect may never execute, or may only fail under specific conditions.

Testing vs. Quality Assurance Ch1

	Testing	QA
Focus	Product	Process
Goal	Find defects	Prevent defects
Orientation	Corrective	Preventive

Trap: "QA and testing are the same thing." — FALSE. Testing is part of QA, but QA is broader — it covers the entire development process.

Verification vs. Validation Ch1

- **Verification** → "Are we building the product **right**?" — conformance to specification
- **Validation** → "Are we building the **right** product?" — fitness for user needs

Memory tip: Validation = Value to the user.

Confirmation Testing vs. Regression Testing ch2

- **Confirmation testing:** re-tests a specific defect that was fixed — did *this* fix work?
- **Regression testing:** re-tests the whole system after any change — did anything *else* break?

Trap: "Regression testing is only needed after bug fixes." — FALSE. Any change (enhancement, refactor, configuration change) can introduce regressions.

Black-box / White-box as Test Types vs. Test Techniques Ch2 Ch4

The same terms appear in two different chapters with related but distinct meanings:

Context	Meaning
Ch2 — Test types	<i>What</i> the tests are based on: specification (black-box) vs. structure (white-box)
Ch4 — Test techniques	<i>How</i> test cases are derived: from spec (black-box techniques) vs. from code (white-box techniques)

Trap: Seeing "black-box" in a question and assuming it always means the Ch4 technique. Check what the question is actually asking about.

Static Testing vs. Static Analysis Ch3

- **Static testing** = the broad category — evaluating a work product *without executing it* (includes reviews AND tool-based analysis)
- **Static analysis** = the tool-based subset of static testing — automated analysis against a formal syntax

Trap: "Static analysis and static testing are the same." — FALSE. Static analysis is *one type* of static testing.

Walkthrough vs. Technical Review vs. Inspection Ch3

	Walkthrough	Technical Review	Inspection
Led by	Author	Moderator	Moderator (NOT author)
Formality	Low–Medium	Medium–High	Highest
Scribe	Not required	Depends	Mandatory
Author as leader?	Yes	No	Never

Trap: "In a walkthrough, individual review beforehand is mandatory." — FALSE. It is *optional* (the syllabus says "not required"). **Trap:** "In a walkthrough, a scribe is mandatory." — FALSE. A scribe is optional in walkthroughs; the author may even fill the role. **Trap:** "In an inspection, the author can lead the review." — FALSE. The author cannot be the review leader or the scribe.

Equivalence Partitioning vs. Boundary Value Analysis Ch4

- **EP** — selects *one representative value* from each partition (any value, not necessarily the edge)
- **BVA** — specifically targets the *edges* of ordered partitions; extends EP, doesn't replace it

Trap: "BVA replaces EP." — FALSE. BVA extends EP by adding boundary-specific test cases on top of it.

2-value BVA vs. 3-value BVA Ch4

- **2-value:** boundary value + its closest neighbor from the adjacent partition
- **3-value:** the boundary value itself + the value immediately below it + the value immediately above it

Trap: "3-value BVA tests three partitions." — FALSE. It tests three specific values around one boundary.

Statement Coverage vs. Branch Coverage Ch4

- **Statement coverage:** every executable statement run at least once
- **Branch coverage:** every branch (true/false outcome of every decision) exercised

Critical rule: Branch coverage **subsumes** statement coverage — 100% branch coverage guarantees 100% statement coverage. The reverse is NOT true. **Trap:** "100% statement coverage means all branches are tested." — FALSE.

Severity vs. Priority Ch5

- **Severity** = how badly the defect impacts the system (technical view)
- **Priority** = how urgently it needs to be fixed (business view)

Trap: "A high-severity defect is always high priority." — FALSE. Example: a crash in a feature used by 0.1% of users = high severity, low priority. Example: a typo on the login page = low severity, high priority.

Test Monitoring vs. Test Control Ch5

- **Monitoring** = collecting data and checking progress against the plan (observing)
- **Control** = taking corrective action when deviations are found (acting)

Trap: "Monitoring and control are the same activity." — FALSE. Monitoring without control is just observation.

Product Risk vs. Project Risk Ch5

- **Product risk** = risk that the system fails to meet quality expectations → drives *testing* decisions
- **Project risk** = risk to the project schedule, budget, or resources → managed by *project management*

Exam Traps — Statements That Look True But Aren't

Trap statement	Why it's wrong
"Testing proves the software is correct."	Testing shows the <i>presence</i> of defects, never their absence (Principle 1)
"With enough time, we can test everything."	Exhaustive testing is impossible (Principle 2)
"If all tests pass, the software has no defects."	Absence of defects is a fallacy (Principle 7)
"More testing always means higher quality."	Testing is context-dependent; more is not always better (Principle 6)
"The author can lead or scribe an inspection."	Not allowed — unique rule for inspections
"Q1 in the testing quadrants is business-facing."	Q1 is <i>technology-facing</i> + supporting the team; Q2 is business-facing
"Regression testing only runs after a bug fix."	Any change — enhancements, refactors, migrations — can break existing behavior
"100% statement coverage guarantees full testing."	It does not test all branches; branch coverage is stronger
"Exploratory testing has no structure."	It uses time-boxed <i>sessions</i> guided by a <i>test charter</i>
"0-switch coverage tests sequences of transitions."	0-switch = single transitions only, no chaining
"White-box testing is always dynamic."	White-box <i>analysis</i> can be applied statically to pseudocode or unexecutable code
"Planning Poker is the only named estimation technique in CTFL 4.0."	CTFL v4.0.1 names four techniques: Estimation based on ratios, Extrapolation, Wideband Delphi , and Three-point estimation. Planning Poker is a <i>variant</i> of Wideband Delphi, not a separate technique.
"A walkthrough requires individual review beforehand."	Individual review is <i>optional</i> in walkthroughs. A scribe is also optional (not mandatory).
"BVA can be applied to any partition."	BVA only applies to <i>ordered</i> partitions (those with min/max values)

Tips to Avoid Traps

Read the question stem carefully. ISTQB questions often hinge on a single word: *BEST*, *MOST appropriate*, *EXCEPT*, *NOT*, *LEAST likely*. Missing that word flips the correct answer.

Know the difference between K-levels in the question. - A K1 question asks you to *identify* or *recall* — pick the correct definition. - A K2 question asks you to *explain* or *distinguish* — pick the most accurate explanation. - A K3 question gives you a *scenario* — apply the technique and derive the result.

When two answers both seem correct, go more specific. ISTQB prefers the answer that matches the syllabus definition most precisely. The "almost right" answer is usually a distractor that swaps one key word.

For K3 technique questions, work it on paper. Don't try to do EP, BVA, or state transition in your head. Sketch the partitions or table quickly — you'll catch errors you'd miss mentally.

The syllabus definition wins over industry practice. If you've worked in QA for years and something "feels" right, but it contradicts the CTFL 4.0 syllabus definition, go with the syllabus. The exam tests the standard, not real-world variation.

Watch for absolutes. Words like *always*, *never*, *all*, *only*, *impossible* are often traps. Testing principles 1 and 2 specifically address what is and isn't possible in testing.

When unsure between two answers, eliminate first. Cross out the two clearly wrong answers. Then compare the remaining two against the exact syllabus wording — the right answer will use the correct terminology.